

Warsaw University of Technology

Faculty of Mathematics
and Information Science

Project Documentation

A system for reconstructing fragmented documents based on the shape and content of their parts

Martyna Sadowska

Hanna Szczerbińska

Zuzanna Zalewska

thesis supervisor

Janusz Rafałko, PhD

WARSAW 2026

1. Change History

Version	Date	Author	Description of Change
1.0	28.10.2026	MS	Baseline version and technology stack
1.1	29.10.2026	MS	First two related work descriptions
1.2	30.10.2026	HS	Second two related work descriptions
1.2	30.10.2026	ZZ	Last two related work descriptions and GUI
1.3	30.10.2026	ZZ	Condensed related work
1.4	30.10.2026	HS	Data reports and component description
1.5	30.10.2026	MS	Sequence diagrams, abstract and reconstruction algorithms proposal
1.6	30.10.2026	ZZ	Splitting algorithms proposal
2.1	31.10.2026	HS	New solution proposal and improved component descriptions
2.2	1.11.2026	MS	Larger font on diagrams

Authors: Martyna Sadowska (MS), Hanna Szczerbińska (HS), Zuzanna Zalewska (ZZ)

Supervisor: Janusz Rafałko, PhD (JR)

Abstract

This document defines the design and architecture of the system. It contains an overview of published works related to the project and data reports for the datasets used in testing. It also includes the solution proposal, component description, technology stack for our architecture and the design of the graphical user interface of the application. The document serves as a shared reference for the project team and supervisor, supporting consistent implementation and a clear solution plan for document reconstruction.

Table of Contents

1 Change History **1**

Abstract **2**

2 Related work **4**

3 Data Reports **5**

3.1 RVL-CDIP 5

3.2 Tobacco800 5

4 Solution Proposal **7**

5 Component Descriptions **7**

5.1 Document Splitting Module 7

5.1.1 Technical Description 7

5.1.2 Document Splitting Algorithms 8

5.2 Document Reconstruction Module 10

5.2.1 Technical Description 10

5.2.2 Document Reconstruction Algorithms 10

6 Technology Stack **12**

7 Graphical User Interface Design **13**

2. Related work

Reconstructing a document from its fragments is mostly treated in literature as a variant of the classic jigsaw puzzle problem. In a survey, authors Yılmaz and Nabiyev [8] demonstrate that existing methods can be classified according to the type of information used to decide whether two fragments fit together, specifically relying on the shape of their contours, the content near their borders, or a combination of both. Most document reconstruction methods they investigate follow the same general pipeline of feature extraction, pairwise scoring of pieces and then assembling the document based on a global overview of the pairwise scores.

Shape-based methods use solely the geometry of the torn edges. Biswas et al. [2] detect the corners of each fragment, divide its outline into segments between such corners, and construct an AVL tree to efficiently search for edges with similar corner-to-corner distances. Later, they also verify the matches between pieces by checking whether, when placed side by side, the borders of both pieces don't diverge too much. Pimenta et al. [7] simplify the fragment contours with the Douglas–Peucker algorithm, describe each vertex by its angle and the distances to its neighbors, and compare the resulting sequences using the longest common subsequence algorithm. The match scores are then included in a graph (where vertices are separate document fragments and edges are the connections between them), and the document is assembled using a graph-based optimisation algorithm which prioritises the strongest matches. These approaches' main advantage is that they work independently of the document content. Their main limitation is that they only work when the document fragments' edges are sufficiently irregular: in particular, such algorithms struggle with fragments produced by linear cuts, which do not contain enough distinctive vertices and angles. This limits their use in cases such as shredded documents.

Zhang et al. [9] combine both sources of information, comparing contour segments using geometric as well as color features. They measure the difference between two segments using the Earth Mover's Distance and assemble the fragments greedily, starting from the most similar pairs. However, its performance decreases for fragments of highly irregular shape, which our system is also expected to handle.

For strip-shredded documents where all edges are straight and identical, content is the only factor which can be used in reconstructing the document. Perl et al. [6] recognize the characters cut through by a strip border and pairs strips in which the parts of letters along cut lines form valid characters when combined, although they find that characters which have been cut are considerably harder to recognize than complete ones. Paixão et al. [5] instead train a convolutional neural network in a self-supervised manner in order to score the compatibility of strip pairs, and simplify the problem of reconstructing the original document into a variant of the traveling salesman problem.

However, both methods only work for vertical strips of known orientation that are already separated from the background.

Therefore, each of the described solutions addresses a single type of fragment. Our system aims to handle a variety of them, including strips, rectangles, and irregular pieces, also supporting rotated pieces. Furthermore, we intend to compare the various shape-based, content-based and combined methods on datasets with various piece shapes and sizes, creating heuristics for the selection of document-reconstruction algorithm that suits the particular set of presented pieces.

3. Data Reports

In our work, we're primarily using two datasets: Tobacco-800 (introduced by Lewis et al. [4] and Agam et al. [1]) and RVL-CDIP (introduced by Harley et al. [3]).

3.1. RVL-CDIP

The dataset consists of 400,000 grayscale images, which are divided into 16 classes, with 25,000 images per class. The document classes include letter, form, email, handwritten, advertisement, scientific report, scientific publication, specification, file folder, news article, budget, invoice, presentation, questionnaire, resume, and memo. Their large variety allows us to test the reconstruction algorithms, which try to connect the document pieces using information about their content, ranging from simple colour-matching on the border to character recognition. The images are sized so their largest dimension does not exceed 1000 pixels. Two example images from this dataset are shown on Figure 1.

3.2. Tobacco800

The dataset contains 1292 files, which are all .png images of typewritten documents. Some of those also contain a limited amount of handwritten text (such as signatures) or tabular data. Resolutions of the documents vary significantly from 150 to 300 DPI, and the dimensions of images range from 1200 by 1600 to 2500 by 3200 pixels. Notably, the images in this dataset are in white-on-black form, which makes it a valuable resource when testing the performance of document reconstruction algorithms in environments other than the standard black-on-white documents. An example image from this dataset is shown on Figure 2.

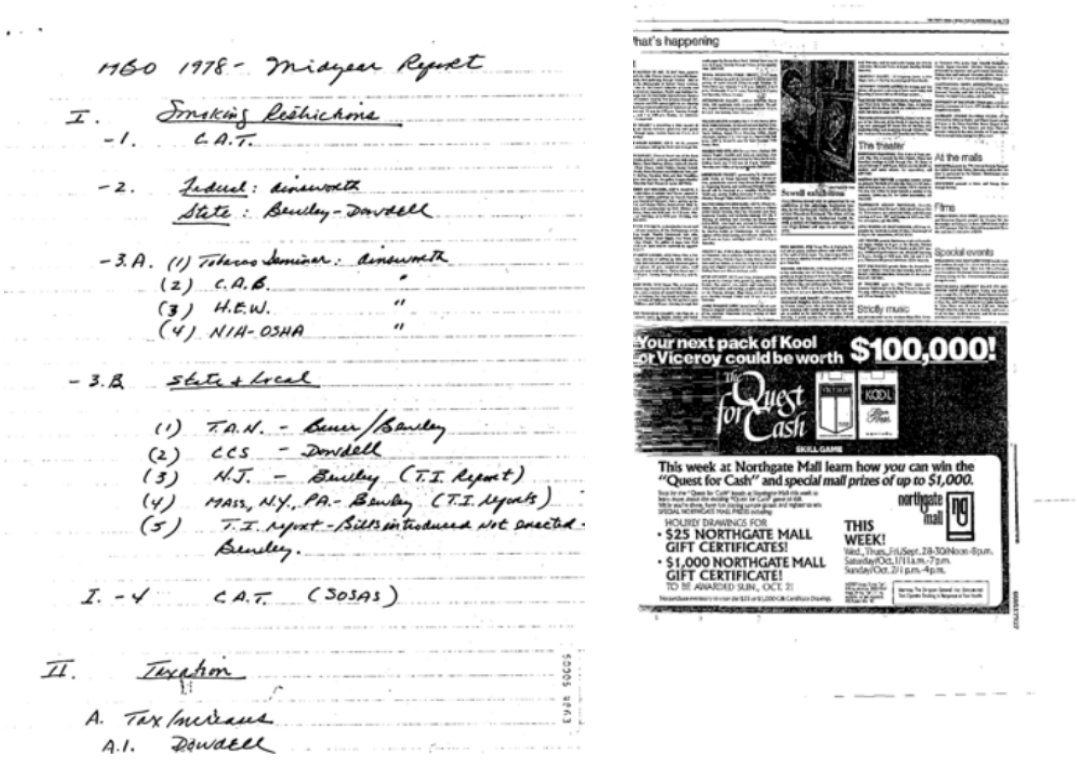


Figure 1: Example images from the RVL-CDIP dataset.

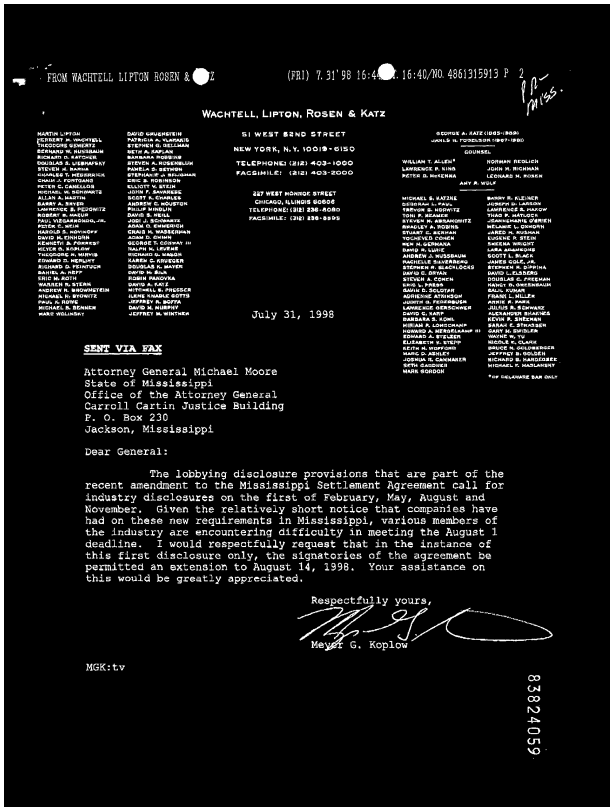


Figure 2: An example image from the Tobacco800 dataset.

4. Solution Proposal

The solution we propose is a desktop application which will support both the generation and reconstruction of fragmented documents. The system is divided into two main functional modules: the Document Splitting Module and the Document Reconstruction Module, both of which operate independently and can be accessed through a shared graphical user interface.

The Document Splitting Module is responsible for generating fragmented documents from original document images. It operates in two main stages. First, using the splitting algorithm selected by the user, it generates a mask representing the cutting lines on the original document and uses it to divide the original image into separate pieces. Second, it arranges the resulting fragments on a background and saves the resulting image.

The Document Reconstruction Module performs the reverse operation: it takes an image containing multiple document fragments and attempts to recover the original document from them. Similar to the Document Splitting module, this one also operates in two main stages. First, the fragment-extraction component identifies and extracts individual pieces from the original image. Second, the document reconstruction component analyses the extracted fragments and attempts to arrange them into a full document.

Both modules follow a similar processing workflow. The user first provides an input image and selects the desired algorithm and its parameters. The application then performs the requested image-processing operations and allows the user to save the generated output. Where applicable, the user can inspect intermediate results (e.g. proposed cutting lines or detected fragment borders) before proceeding with the main operation. This approach provides greater control over the processing pipeline and makes it easier to select an algorithm most suitable for the user's needs. Together, the two proposed modules provide a complete environment for experimenting with document fragmentation and reconstruction techniques.

5. Component Descriptions

5.1. Document Splitting Module

The purpose of the first module is to split a picture of a document into pieces, returning an image with those pieces arranged on a background. By supporting multiple splitting strategies (including both regular and irregular piece shapes), this module enables the generation of diverse test cases for evaluating document reconstruction algorithms.

5.1.1. Technical Description

In this module, the user will initially be able to select an image from their file system and load it into the app (responding to the functional requirement described in user

story US-01). After a document is loaded, the image is automatically converted into grayscale and displayed to the user. The user can then select an algorithm for splitting the document into pieces (i.e. their preferred piece shapes and the method of generating them), as well as its parameters such as the desired number of pieces; initially, a default algorithm and its parameters are suggested. After the parameters have been set, the user has the option to preview the document split. Upon selecting it, the program generates an image mask which contains the fragment borders and displays it to the user (fulfilling the functional requirement described in US-02). After that, the user can run the algorithm which splits the document to pieces and returns an image of the fragments semi-randomly arranged on a background. Alternatively, the user can also select to split the image without previewing pieces – in this case, the mask gets generated automatically before the document is split. At the end of this process, the final image is shown to the user and automatically saved into their chosen disk location (fulfilling the functional requirement described in US-03).

This process is also illustrated by its dedicated sequence diagram (3), which explains the process for single mode image processing. In batch mode, there are some minor differences: only the first document is available for preview, and all of the steps performed by user are done for all consecutive documents as well, saving each one right after it gets processed.

5.1.2. Document Splitting Algorithms

This module supports a variety of cutting strategies, generating fragment sets that reflect different real-world shredding and tearing techniques. Each generated fragment is represented as a rectangular matrix where pixels outside the fragment boundary are rendered fully transparent. Pixels falling directly on the calculated split lines are either randomly assigned to one of the neighboring fragments or discarded entirely to simulate material loss such as paper dust or the width of a shredder blade.

The implemented algorithms range from simple geometric partitions to complex shapes. Regular fragmentation methods include both uniform and randomized vertical or horizontal strips. For irregular partitions, the system utilizes geometric methods such as random straight cuts and Voronoi diagrams. Furthermore, in order to replicate natural paper degradation, the module supports jagged edge generation with random displacements.

In addition to the cutting geometry, the module enforces practical constraints such as defining minimum fragment dimensions to prevent too small or narrow pieces and setting strict limits on maximum piece counts.

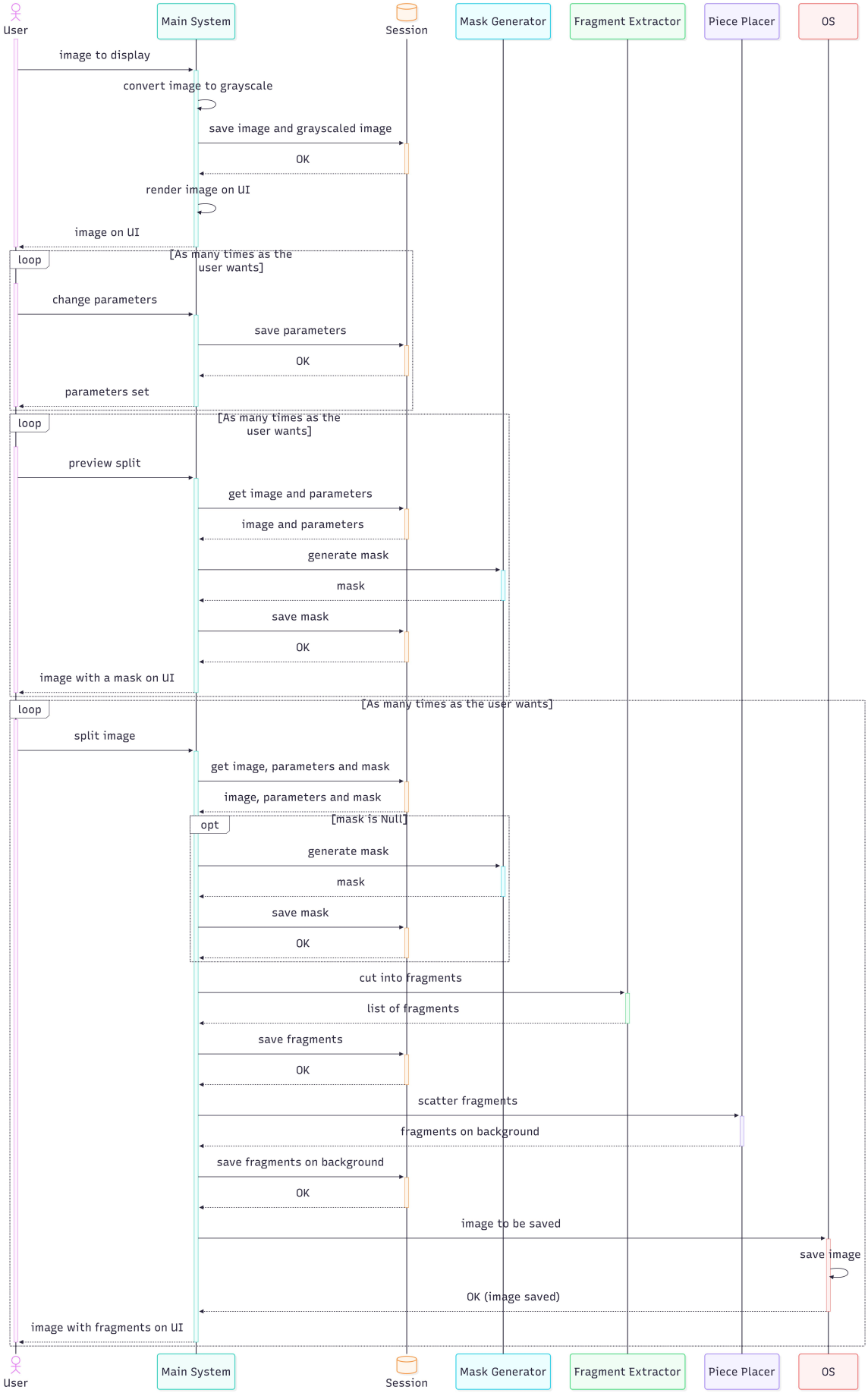


Figure 3: Sequence diagram for splitting a document in single mode.

5.2. Document Reconstruction Module

The purpose of the second module is to reconstruct a document from an image of its pieces. It will support various fragment-extraction and document-reconstruction algorithms in order to maximize the chance of successfully reconstructing a document and enable the comparison between various algorithms.

5.2.1. Technical Description

In this module, the user will initially be able to select an image from their file system and load it into the app. After a document is loaded, the image is automatically converted into grayscale and displayed to the user. The user can then select an algorithm for extracting individual fragments from the image; initially, a default algorithm and its parameters are suggested.

After the parameters have been set, the user has the option to run the algorithm and preview the fragment borders. Upon selecting it, the program generates an image mask which contains the fragment borders and displays it to the user (fulfilling the functional requirement described in user story US-04). Additionally, a list of smaller images (each containing a single document background on a transparent background) is created and temporarily saved.

After that, the user can move on into document reconstruction section. Here, they can select their preferred algorithm for reconstructing the document, along with its parameters (the default, best-performing algorithm and parameters are also pre-selected at this stage). Upon running the algorithm, the user can preview the reconstructed document (fulfilling the functional requirement described in user story US-05) and either decide to save the result or re-run the reconstruction process with either a different algorithm or its parameters (fulfilling the functional requirement described in user story US-06). When they're satisfied with the result, the user can save the image to their chosen disk location.

This process is also illustrated by its dedicated sequence diagram (4), which explains the process for single mode image processing. In batch mode, there are some minor differences: only the first document is available for preview, all of the steps performed by user are done for all consecutive documents as well, and each document is saved right after it gets processed, meaning that the user cannot iteratively improve the result without saving it in-between runs.

5.2.2. Document Reconstruction Algorithms

In this module, we intend to test various reconstruction algorithms relying on the fragments' shape, content and both of those at the same time. We will also aim to combine approaches from literature that rely only on one of those characteristics to create a more versatile algorithm.

For our shape-based approach, we will extract and calculate multiple pieces' features,

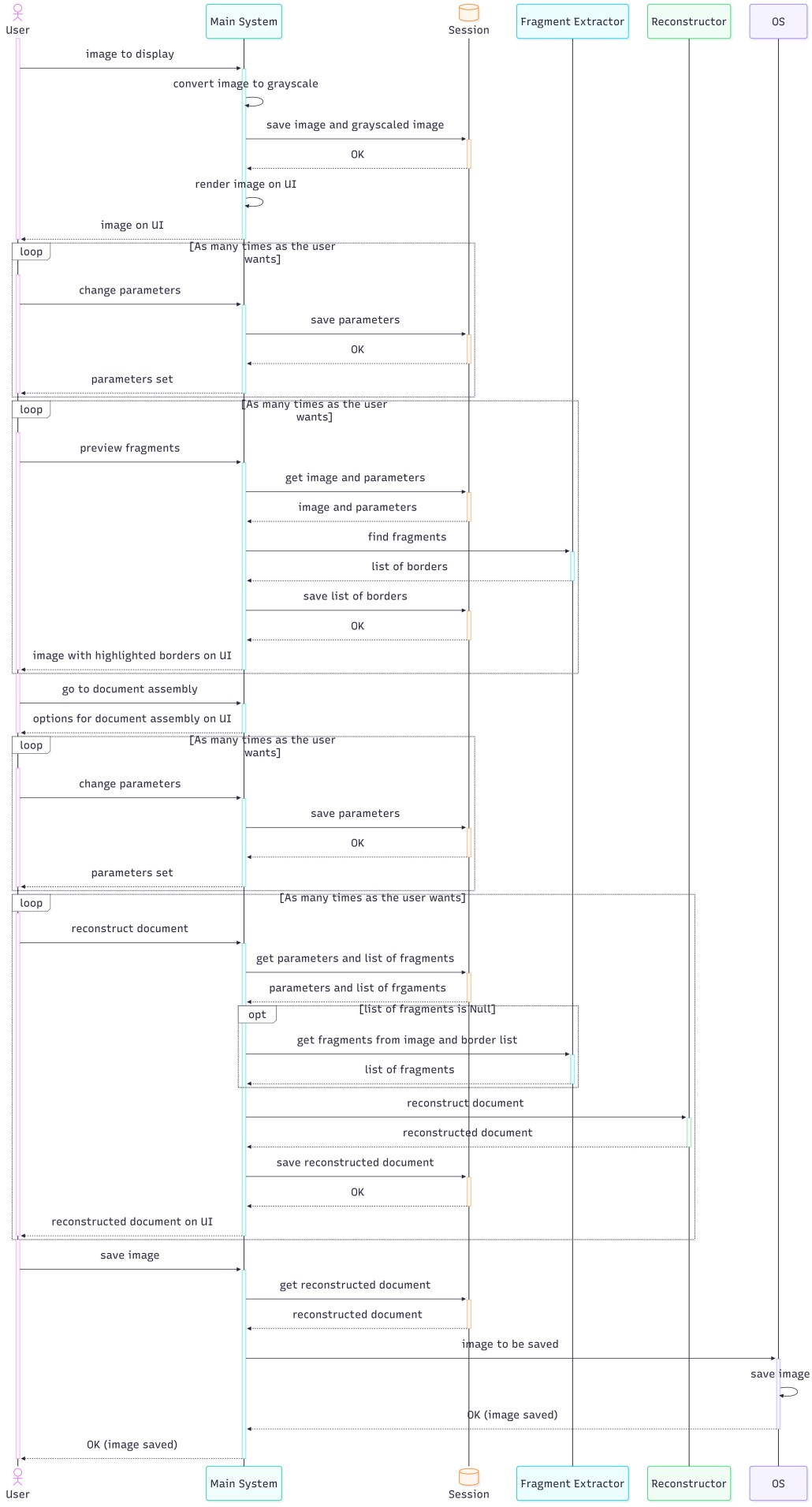


Figure 4: Sequence diagram for reconstructing a document in single mode.

including their length, the Euclidean distance between them, the chain codes representing the edges' shape, and the angles of their vertices. Our content-based analysis will focus on analysing the fragments' borders and finding similarities through calculating gradients, differences in colour values, or OCR. To utilise information from both sources, we plan to combine those values into one comprehensive measure.

To find appropriate matches based on these measures, we will implement standard methods like AVL trees or dynamic programming. We will also test the performance of machine learning algorithms such as convolutional neural networks and evolutionary algorithms.

It is important to note that our main goal in reconstructing the documents is to test a number of different reconstruction algorithms and try to create heuristics that will allow the program to choose a particular algorithm and its parameters that suit the presented pieces best. To achieve that, we will benchmark the algorithms with different sets of fragments based on metrics like the proportion of correctly connected pieces, distances in an embedding space, OCR analysis and manual visual inspection. Additionally, we will synthesise findings from the related work to establish the theoretical advantages of each algorithm, such as their performance on vertical stripes or fragments with irregular edges. This would allow the program to select the best reconstruction algorithm for a given document's pieces.

6. Technology Stack

Module / Component	**Selected Technology**	**Considered Alternatives**	**Justification and Trade-offs**
Backend	**Python**	Java	Provides integration with computer-vision libraries and enables rapid prototyping. The main trade-off is lower raw execution speed.
UI	**Custom Tkinter**	Tkinter, PyQT6	Combines a low learning curve with greater design flexibility compared to standard Tkinter.
Image processing	**OpenCV**	Pillow, Scikit-Image	A highly optimized image-processing library that will speed-up our whole process. It provides both simple functionalities like rotation, filtering or segmentation, but also feature detection and extraction, which is crucial for analyzing and matching shredded fragments.

Module / Component	Selected Technology	Considered Alternatives	Justification and Trade-offs
Displaying images on GUI	**Pillow**	OpenCV	OpenCV's native image display does not integrate into CustomTkinter interfaces. Pillow acts as a bridge, converting OpenCV's arrays into Tkinter-compatible objects.
Matrix operations	**Numpy**	Native Python lists	Crucial for efficient multidimensional array operations. Since OpenCV natively represents images as Numpy arrays, it is necessary for fast, pixel-level manipulation.
Deployment	**PyInstaller**	Nuitka	Packages the application and all its dependencies into a single, standalone executable (.exe) file. This ensures end-users can run the application without installing Python or configuring virtual environments.
OCR (Optical Character Recognition)	**Python Tesseract**	Azure OCR, IronOCR	Open-source and runs entirely offline, which allows for local, secure benchmarking for the reconstruction algorithms. The trade-off is slightly lower accuracy on heavily distorted text compared to commercial cloud APIs.
Testing	**Pytest**	unittest (built-in)	Allows us to write simple test functions with detailed error messages and without the heavy class structure required by the built-in *unittest*.

7. Graphical User Interface Design

The application runs in a single window shared by both modules, divided into three areas: a top bar with the file actions menu and the module switch, a preview area with the image being processed and a parameter panel in which it is possible to change the algorithm and its parameters. The interface dynamically adapts to the active module and processing stage, displaying only the operations available at this stage.

The cutting module divides a document into fragments and arranges them on a background. In the preview area, the original file is first displayed; then, after the user generates a split

mask for the document, it is overlaid on top of the original document in the preview section. When the user runs the splitting algorithm and the final image is saved to their chosen disk location, the preview area displays the final image with pieces arranged on a background. The parameter panel originally displays the default image-splitting algorithm with default parameters; once the user selects a different algorithm, the default parameters for this algorithm are shown. The user can then use the panel to modify any of the parameters, and then choose the 'Preview Split' or 'Run' option. The UI mock-up for this module is shown in Figure 5.

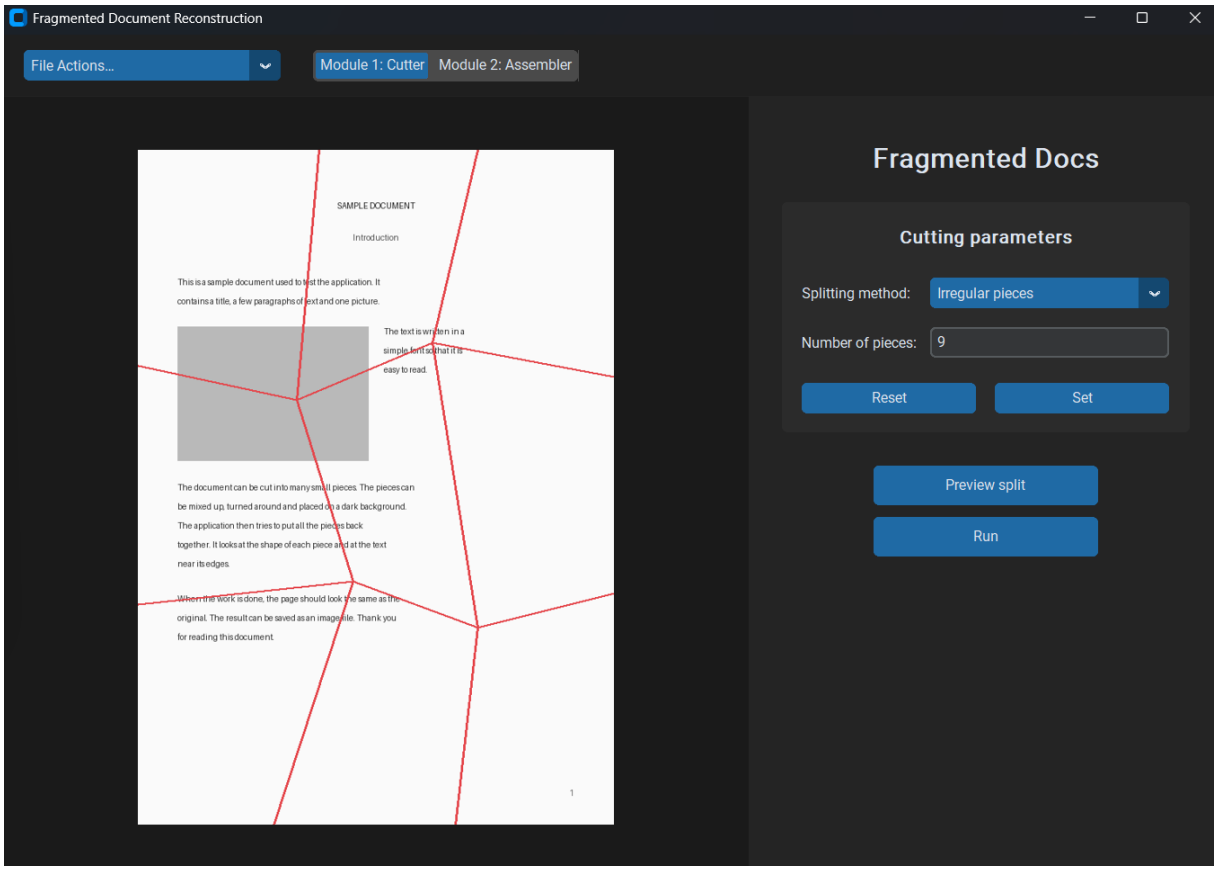


Figure 5: Cutting module screen with a preview of the split lines

The assembly module reconstructs a document from a photo of its fragments. The content displayed on the UI changes based on whether the user is at the stage of retrieving individual fragments from the image, or reconstructing the original document.

Originally, the preview panel displays the image of fragments supplied by the user, and the parameter panel allows the user to select their preferred fragment-extraction algorithm and preview detected fragment borders. After the user selects the 'Show Fragment Borders' button in the parameter panel, the preview panel displays their original image with the fragment borders overlaid on it.

After the user moves to the document assembly section, the parameter panel allows them to select an algorithm and its parameters, with the default options pre-set in the selection fields. After they run the selected algorithm, the preview panel displays the assembled document. The UI mock-up for this module is shown in Figure 6.

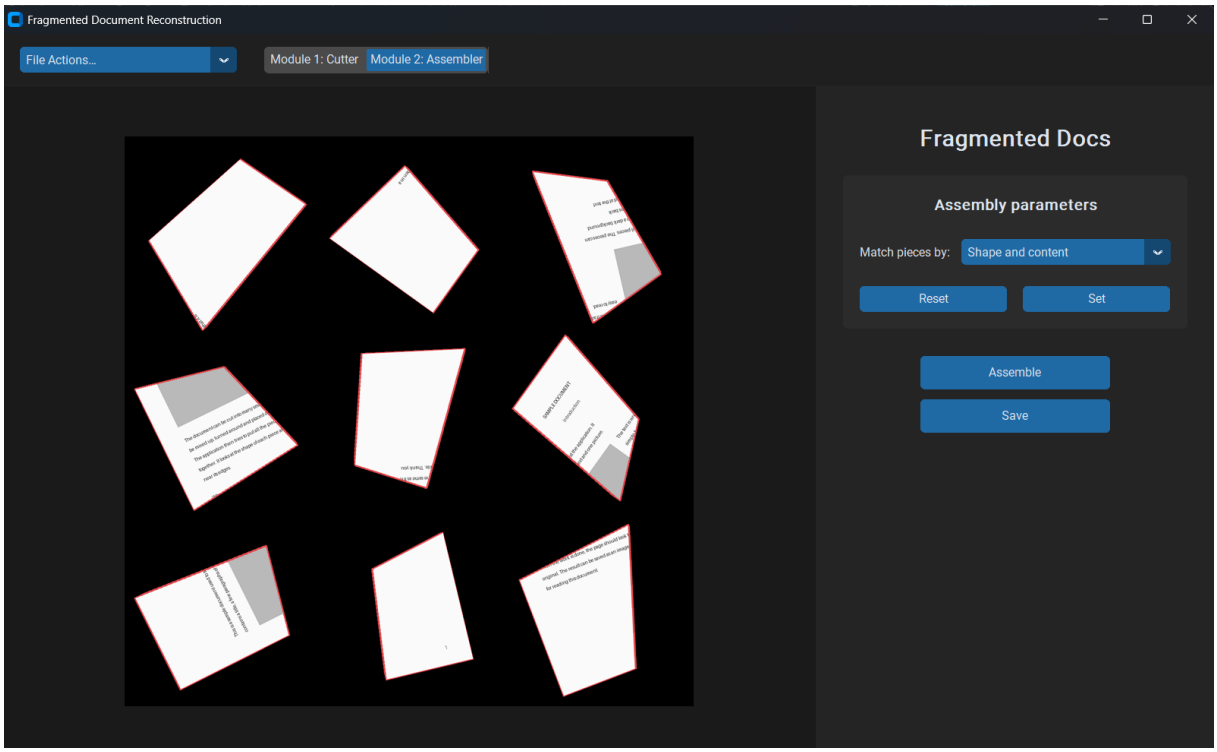


Figure 6: Assembly module screen

References

[1] Gady Agam, Shlomo Argamon, Ophir Frieder, David Grossman, and David Lewis. The complex document image processing (cdip) test collection project. *Illinois Institute of Technology*, 2006.

[2] Arindam Biswas, Partha Bhowmick, and Bhargab B. Bhattacharya. Reconstruction of torn documents using contour maps. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, 2005.

[3] Adam W Harley, Alex Ufkes, and Konstantinos G Derpanis. Evaluation of deep convolutional nets for document image classification and retrieval. In *2015 13th international conference on document analysis and recognition (ICDAR)*, pages 991–995. IEEE, 2015.

[4] David Lewis, Gady Agam, Shlomo Argamon, Ophir Frieder, David Grossman, and Jefferson Heard. Building a test collection for complex document information processing.

In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 665–666, 2006.

[5] Thiago M. Paixão, Rodrigo F. Berriel, Maria C. S. Boeres, Alessandro L. Koerich, Claudine Badue, Alberto F. De Souza, and Thiago Oliveira-Santos. Self-supervised deep reconstruction of mixed strip-shredded text documents. *Pattern Recognition*, 2020.

[6] Johannes Perl, Markus Diem, Florian Kleber, and Robert Sablatnig. Strip shredded document reconstruction using optical character recognition. In *Proceedings of the 4th International Conference on Imaging for Crime Detection and Prevention (ICDP)*, 2011.

[7] Andre Pimenta, Edson Justino, Luiz S. Oliveira, and Robert Sabourin. Document reconstruction using dynamic programming. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2009.

[8] Seçkin Yılmaz and Vasif V. Nabiyev. Comprehensive survey of the solving puzzle problems. *Computer Science Review*, 2023.

[9] Meng Zhang, Shuangmin Chen, Zhenyu Shu, Shi-Qing Xin, Jieyu Zhao, Guang Jin, Rong Zhang, and Jürgen Beyerer. Fast algorithm for 2D fragment assembly based on partial EMD. *The Visual Computer*, 2017.